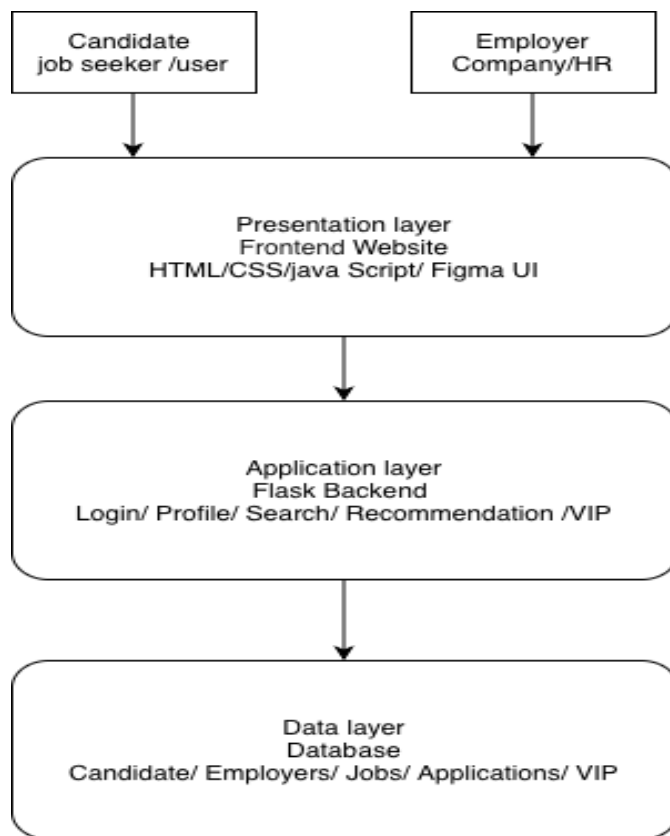
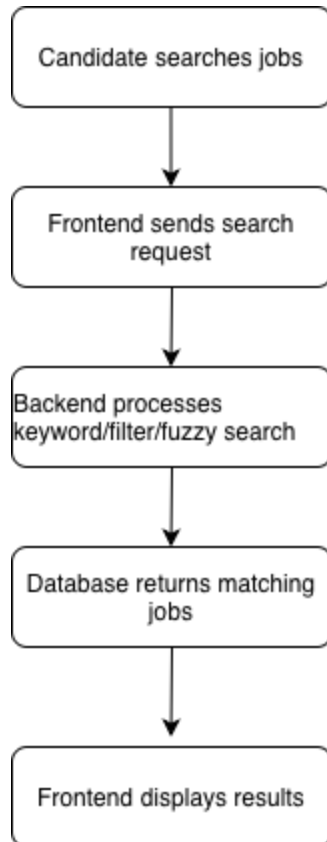


1. Introduction

The Job Radar platform adopts a three-layer architecture consisting of the presentation layer, application layer, and data layer, which separates the user interface, business logic, and data storage, thereby improving the system's maintainability and scalability. The system is primarily designed for two types of users—job seekers and employers—who interact with the system through a web interface, while the backend handles requests and stores data in the database.

2. System Architecture Diagram





Presentation Layer

The presentation layer provides a user interface for job seekers and employers. Job seekers can register, log in, manage their profiles, search for jobs, view recommended information, and track their application progress. Employers can create company profiles, post jobs, search for candidates, and view recommended applicants. The interface is built based on the final Figma design, and users can access the system's features through a web browser.

Application Layer

The application layer is implemented using the Flask framework and handles the core business logic of the system. It processes user requests, manages authentication, handles job and candidate searches, supports profile management, and controls the

recommendation features. The application layer also manages membership functions and coordinates the data exchange between the user interface and the database.

Data Layer

The data layer is responsible for storing all system information in the database, including candidate profiles, employer profiles, job postings, application records, and membership information. This layer provides persistent storage capability, allowing the system to efficiently retrieve and update the required information.

Components interaction

When a user submits a request through the website, the request is sent to the Flask backend for processing. The backend then retrieves or updates data from the database based on the request and returns the results to the user interface. For example, when a job seeker searches for jobs, the frontend sends the search request to the backend, and the backend retrieves the matching job information from the database and displays the results on the website.